tick2trade-fpga: Architecture Specification (v3)

Riya Rathod · Oct 28, 2026 · Status: plan only, nothing built

This is the single source of truth for building tick2trade-fpga on top of tmlee06/Itch-Parser. Anything marked [N] is normative: code or a test that disagrees with it is wrong, or this spec must be changed first.

Provenance of this revision. Sections 1 and 2 are the approved text. Sections 3 to 16 were written after reading every RTL, testbench and data file of the upstream repository on Oct 28, 2026. Nothing was executed. Anything in section 3 marked [VERIFY] comes from reading rendered source and must be confirmed by a run at T0. Where this pass had to choose a behaviour the owner has not approved, it is listed in section 16.3 (C-choices) and stays normative until the owner changes it.

1. How to use this spec

Tags. [N] normative. [I] informative (a recommendation the implementer may change). [VERIFY-n] a fact about the upstream repo or a tool that must be confirmed by reading it at T0. [TBD-n] a choice the owner has not made yet (register in section 16).

Precedence. This spec, then committed code, then the upstream README. The README claims (under 840 ns, about 700 MHz, 11 LUTs at 100 MHz) were never reproduced and must not be copied into this project.

Rules for whoever writes the code (human or Claude):

1. Read sections 3, 4 and the section of the module being built before writing anything. Never guess a port, width or byte offset: it is in this document or it is a [TBD].
2. One task (section 15) per change set. The commit message starts with the task id, for example `T5: order table bypass`.
3. Never edit anything under `upstream/`. Wrap or re-implement instead.
4. Interface changes (ports, widths, formats) are made in this spec first and in code in the same commit.
5. Every normative behaviour has a test id in section 12. A module is done only when its listed tests pass and their logs are committed to `reports/sim/`.
6. A number in a README, resume line or commit message must link to a file in `reports/` plus a commit hash, and carry exactly one label: measured, bounded-proven, reported, computed or estimate (section 13).

7. If a gate fails, report the real number and open a task to fix the design. Never relax a gate silently.
8. No result may be claimed for a part it was not run on. In particular, no U55C result unless a U55C run produced it.

When this spec is silent, pick the simplest fail-closed behaviour (no order is emitted), record it in section 16, and ask the owner.

2. Resolved decisions and assumptions

2.1 Decisions

ID	Decision [N]	Consequence
D1	Default and only claimed target: Artix-7 `xc7a35tcpg236-1` (Basys 3), build profile `A7`. Confirm once with `get_parts xc7a35t*` in the Vivado Tcl console.	All Fmax, WNS and utilisation numbers come from this part. The Alveo U55C (`xcu55c-fsvh2892-2L-e`) appears only as a future-port note in the README; its profile parameters exist in section 4 so a port is mechanical.
D2	Upstream license is unresolved. Treat upstream files as all rights reserved until the author confirms in writing.	See 3.3. Do not publish the repo or send it to Zanskar until permission is recorded in `CREDITS.md`.
D3	8 symbols by default (`N_SYMBOLS=8`), configurable at build time. Slots 0 to 2 are AAPL, SPY, MSFT (the tickers upstream already tested).	Slots 3 to 7 are chosen by the rule in 8.1 from the sample data.
D4	Outbound format is called OUCH-style everywhere (docs, comments, README). Never write OUCH without the suffix.	The frame layout is defined by this spec (6.2), not by the NASDAQ OUCH document. Interview line: NASDAQ formats are used because they are open and well documented, and the same structure applies to NSE binary protocols.
D5	Gates are in section 13. Four changes from v2: throughput gate sits at the book-core input, Fmax gate depends on the part, coverage is line plus toggle, the independent ITCH cross-check records file name and hash.	Label scheme (measured, bounded-proven, reported, computed, estimate) is unchanged.

ID	Decision [N]	Consequence
D6	New repository `tick2trade-fpga`, upstream credited, not a fork.	Upstream files are copied unmodified into `upstream/`.
D7	Pipeline order: ingest, framer, extract, filter, table, book, trigger, risk, encoder.	Risk needs a candidate order, and only the trigger makes one.
D8	Book policy: top-8 levels per side plus a hidden-order count (section 9).	Staleness is derived, never guessed.
D9	RTL is synthesizable Verilog-2005 (matches upstream) with flat packed vectors and one shared header `rtl/t2t_defs.vh`. Testbenches are SystemVerilog under Verilator 5.x; plain Verilog tests also run under Icarus. Model and tooling are Python 3.10 or newer. Yosys for estimates.	No SystemVerilog interfaces or structs in `rtl/`.

2.2 Assumptions

ID	Assumption	Status
A1	The stream entering the core is the MoldUDP64 payload only: a back-to-back sequence of [len16 big-endian] [message] blocks. A frame (first beat to `tlast`) always holds whole blocks, never a split message.	[VERIFY-1] against how upstream `top_parser_to_lab.v` and the adapter are fed
A2	The stream-side source cannot be stalled (no `tready`).	Matches the CMAC receive interface; stands for all profiles.
A3	A session starts from an empty book (ITCH day files start before any order exists).	The only exact recovery path is session reset (4.4).
A4	The longest ITCH 5.0 message is 50 bytes (stored length, excluding the 2-byte prefix).	[VERIFY-2] against the NASDAQ TotalView-ITCH 5.0 specification at T1
A5	A real NASDAQ ITCH 5.0 sample file can be downloaded.	[VERIFY-3] at T1; otherwise the owner supplies the file (section 16)

ID	Assumption	Status
A6	Byte lane 0 of `tdata` (`tdata[7:0]`) is the first byte on the wire, on both rx and tx.	Taken from the upstream adapter header comment citing Xilinx PG203. [VERIFY-10]
A7	Everything runs in one clock domain `clk` : rx stream, tx stream and AXI-Lite.	Matches upstream (`ap_clk` everywhere). No CDC logic exists in this project.

2.3 What this version supersedes in the approval plan

Approval plan said	v3 rule
Sticky stale flag, cleared by an explicit rebuild	Derived `stale` (hidden count) plus a separate sticky `corrupt` flag cleared only by session reset (D8, section 9)
Fmax gate 4.000 ns on U55C	Part-dependent gate (section 13, G6)
Coverage: line, branch, FSM states	Line plus toggle; FSM state coverage as functional bins; branch not claimed (G8)
Throughput 1 msg/cycle at the decoder	Gate at the book-core input; end-to-end reported separately (G4)
Add MIT text to `upstream/`	Forbidden. Upstream files carry no license text from us (3.3)
Four labels: measured, proven, reported, computed	Five labels: measured, bounded-proven, reported, computed, estimate (13.2)
T0 to T12	T0 to T13: the old T4 is split into T4 (filter) and T5 (table). All later ids shift by one (section 15)

3. Upstream audit (tmlee06/Itch-Parser)

Canonical name: `tmlee06/Itch-Parser` (the URL also resolves as `itch-parser`). The branch read was `main` , which showed 4 commits. The commit hash was not visible in the rendered pages: T0 records `git rev-parse HEAD` in `upstream/UPSTREAM.md` [VERIFY-5].

3.1 File inventory

File	Size	Module and interface	v3 use
`README.md`		claims only	Not copied. Claims excluded (section 1)
`itch_decoder.v`	95 lines	`itch_decoder`: in `clk, rst, data_in[7:0], valid_in`; out `msg_type[7:0], order_shares[31:0], order_price[31:0], order_valid`. Async reset (`posedge rst`)	Characterised at T0. A fixed copy becomes the reference-mode decoder (T3)
`order_book.v`	30 lines	`order_book`: in `clk, rst, incoming_msg_type[7:0], incoming_shares[31:0], incoming_price[31:0], incoming_valid`; out `best_bid_price[31:0], best_ask_price[31:0]`	Not used. Placeholder
`axis_512_to_8_adapter.v`	117 lines	`axis_512_to_8_adapter #(DATA_W=512, KEEP_W=64)`: AXIS slave in, `byte_out[7:0]`, `byte_valid` out. Sync reset	T0 repro and reference-mode harness only. Not in the core path (F7)
`top_parser_to_lab.v`	48 lines	defines module `top_parser_to_lob` (name differs from file), wires decoder to book	Not used
`hft_trading_kernel.v`	154 lines	`hft_trading_kernel`: Vitis wrapper, 512-bit rx and tx AXIS, 12-bit AXI-Lite `s_axi_ctrl_*`	Its port list is the compatibility contract for `rtl/t2t_kernel.v` (11.4). Behaviour not carried

File	Size	Module and interface	v3 use
`tb_axis_adapter.v`	144 lines	three beats: 64 + 64 + 10 bytes, prints PASS when 138 bytes arrive	T0 repro (CHAR-01)
`tb_itch_decoder_real_data.v`	66 lines	reads `sim_data.hex` via `$readmemh`, streams 1 byte per 10 ns, no checks	T0 repro (CHAR-02)
`sim_data.hex`	720 lines, 1 byte per line	data	CHAR-03; regenerated as a valid vector at T1
`hft198slides.pdf`, `index.html`, `.vscode`, `.DS_Store`		not read for RTL facts	Not copied. T0 reads `index.html` and the slides and lists any claim that must stay out

The README names `package_xo.tcl`, `kernel.xml`, `connectivity.cfg` and `synth_only.tcl`. None of them is in the repository listing, so the Vitis packaging flow and the 11 LUT synthesis run cannot be reproduced from this repo (F13).

3.2 Findings

ID	Finding (from reading rendered source; nothing executed)	Consequence
F1	README says MIT. No `LICENSE` file is in the listing.	D2, 3.3
F2	`itch_decoder.v`: the combinational `next_state` goes to IDLE on the last byte (`byte_count == msg_length-1`), and the datapath is a `case (next_state)`. So the `READ_PAYLOAD` branch, which holds the price capture and the `order_valid <= 1`, does not run for the last byte. By this reading `order_valid` never pulses. [VERIFY-4]	The upstream decoder cannot be used as a reference unchanged

ID	Finding (from reading rendered source; nothing executed)	Consequence
F3	Same cause: for a 36-byte Add Order the last byte (offset 35, `order_price[7:0]`) is never captured. [VERIFY-4]	Reference-mode copy fixes both (T3)
F4	The decoder handles only type `0x41`, and outputs only shares and price. No side, reference, symbol, locate or timestamp. Offsets used (counted from the type byte): shares 20 to 23, price 32 to 35. These are correct for ITCH 5.0 Add Order.	Offsets reused in 6.1
F5	`order_book.v` assigns each register to itself under a placeholder comment. Both outputs stay 0 forever.	Book is entirely new work
F6	`top_parser_to_lab.v` contains `module top_parser_to_lob`. The kernel and the testbench instantiate `top_parser_to_lob`.	Never rely on file name = module name inside `upstream/`
F7	Adapter: `s_axis_tready = ~draining`, and draining lasts 64 cycles per beat, so at most one beat per 65 cycles (about 0.98 byte per cycle). It ignores `tlast`. It violates A2 because a CMAC cannot be stalled.	New wide ingest (7.1); adapter kept for reference mode only
F8	`tb_axis_adapter.v` checks only the byte count (138). Byte values are printed, never compared.	PASS there means count, not content
F9	`tb_itch_decoder_real_data.v` never reads `best_bid_price`, `best_ask_price` or `order_valid`. The reported 7520 ns equals 100 + 20 + 720 x 10 + 200 ns, the stimulus length.	It is not a latency figure and not a correctness result

ID	Finding (from reading rendered source; nothing executed)	Consequence
F10	`sim_data.hex` as rendered: 720 lines. 20 blocks, each beginning `00 24 41`, and each holding one price (20 distinct `5a xx xx` tails), so 720 / 20 = 36 bytes per block. The length field says 0x24 = 36, which by the ITCH convention excludes the 2-byte prefix, so a valid block is 38 bytes. All blocks are type `A`. Stock, reference, side and locate bytes are 0. Only shares and price vary. [VERIFY-6]	The README claims about AAPL, SPY, MSFT, Execute, Cancel, Delete and real NASDAQ data are not supported by the file. A decoder that trusts the length field would desync after block 1. Regenerated at T1
F11	Kernel AXI-Lite stub: `s_axi_ctrl_bvalid` tied to 1, writes ignored, `aready` lags `arvalid` by a cycle, reads give best_bid at `0x010`, best_ask at `0x018`, `0xDEADBEEF` otherwise. TX tied to 0. `rst_sync = ~ap_rst_n` with no synchroniser despite the comment.	Not carried. Compatible read offsets kept (11.2)
F12	Kernel comment: clock nominally 322 MHz on U55C. Decoder comments say 100 MHz+. No clock was measured anywhere.	No frequency claim inherited
F13	Files named in the README are absent (see 3.1).	Vivado flow rebuilt from scratch (T13)
F14	README synthesis result (11 LUTs, 10 FFs, 0 BRAM) matches logic whose outputs go nowhere.	Not valid evidence; everything is remeasured

3.3 License and credits [N]

1. `upstream/` holds the upstream files byte-for-byte. T0 adds `upstream/UPSTREAM.md` (source URL, commit hash, date copied, SHA-256 of each file). No LICENSE or header is added to those files.
2. The owner opens an issue or emails the author: extending as a learning project with full credit; asks him to add a LICENSE or confirm permission in writing. The reply (text or link) is pasted into `CREDITS.md`.
3. Until that reply is recorded: repo stays private, nothing goes to Zanskar, and `rtl/ref/itch_decoder_ref.v` (the one derived file) is marked derived-pending-permission in `CREDITS.md`.

4. New files carry the owner's copyright line and MIT [TBD-3: or Apache-2.0].
5. `CREDITS.md` has three lists: files that are his (unmodified), files that are yours, and files derived from his with the exact edits made.

3.4 Reuse map [N]

Upstream item	Becomes
Add Order offsets, 2-byte BE length framing	Section 6.1 and the framer
`itch_decoder.v`	`rtl/ref/itch_decoder_ref.v`: same ports, last-byte defect fixed, reference mode only
`axis_512_to_8_adapter.v`	Feeds the reference-mode decoder in `tb/sv/tb_refmode.sv`; the harness respects its `tready`
`hft_trading_kernel.v` port list	`rtl/t2t_kernel.v` ports (same names and widths)
`sim_data.hex`	`vectors/legacy/` raw copy plus a regenerated valid-framing version
Both upstream testbenches	Run unmodified at T0; characterisation testbenches are new files in `tb/`

4. Global conventions

4.1 Build profiles [N]

Selected by one define in `rtl/t2t_defs.vh`: `T2T_PROFILE_A7` (default) or `T2T_PROFILE_U55C`.

Parameter	A7	U55C (future port, no claims)
Part	`xc7a35tcpg236-1`	`xcu55c-fsvh2892-2L-e`
Implementation	Vivado out-of-context synth + impl, no pins, no bitstream	same
Period sweep (ns)	5.000 then 6.667	4.000

Parameter	A7	U55C (future port, no claims)
`N_SYMBOLS` (max 16)	8	8
`BOOK_N` levels per side	8	8
`TABLE_AW` (sets = 2^AW)	9	12
`TABLE_WAYS`	4	4
Order capacity	2048	16384
`ING_DEPTH` (beats)	64	64
`FA_DEPTH`, `FB_DEPTH` (entries)	32	32
`TXF_DEPTH` (frames)	16	16

Fixed widths for all profiles: symbol index 4 bits (`T2T_SYM_W`), level count 4 bits, price 32, quantity 32, orders per level 16 (`T2T_CNT_W`), hidden count 20 (`T2T_HID_W`), message index 32.

4.2 `rtl/t2t_defs.vh` contents [N]

The header holds: the profile defines and parameter values above; ASCII type constants (A `8'h41` , F `8'h46` , E `8'h45` , C `8'h43` , X `8'h58` , D `8'h44` , U `8'h55`); side constants (B `8'h42` , S `8'h53`); op codes (ADD 0, REDUCE 1, CORRUPT 2); rule ids (CROSS_BUY 0, CROSS_SELL 1, IMB_BUY 2, IMB_SELL 3); risk reason codes (see 10.2); and every bus slice macro from 5.2 as `T2T_<BUS>_<FIELD>_HI` and `_LO` . No RTL file may hard-code a width or offset that exists in this header.

4.3 Clocking, reset, elasticity [N]

1. One clock `clk` , rising edge. Reset `rst` is synchronous, active high, and shallow (a registered reset fan-out is allowed). `t2t_kernel` converts `ap_rst_n` to `rst` through a two-flop synchroniser.
2. Every module output is registered.
3. Two handshake classes only. **H1** valid-only, fixed latency, never stalls. **H2** FIFO with an almost-full flag and a credit margin.
4. The elastic points are exactly: `ING` (before the framer), `FA` (between filter and table), `FB` (between table and book), `TXF` (after the encoder). Every other stage is H1.

5. Credit rule: the stage that pops or issues into an H1 chain stops issuing while the downstream FIFO `afull` is set. `afull` asserts at occupancy >= depth - margin, with margin >= (register depth of the unstoppable chain after the issue point) + 2.

FIFO	Written by	Issue point that obeys `afull`	Unstoppable depth	Margin
`ING`	rx stream	none (A2). A write when full drops the beat and raises `ingest_overflow`		
`FA`	S4	framer issue (S2, S3, S4 behind it)	4 + 2 + 2	12
`FB`	S5	table pop from `FA` (S5 behind it, 2 ops per message)	8	12
`TXF`	S9	none. Full means `tx_overflow` and `tx_fault` (11.1)		

6. Any write into a full `FA`, `FB` or `TXF` is a design error: it raises `fatal_overflow`, which sets `session_invalid`, and the testbench fails on it.
7. FIFOs are first-word-fall-through with a registered read. Minimum traversal is 2 cycles. Occupancy is a power-of-two depth.

4.4 Session reset and invalidation [N]

Flag	Scope	Set by	Effect	Cleared by
`session_invalid`	global	`ingest_overflow`, `frame_err`, `keep_err`, `bad_len` on a handled type, `fatal_overflow`	Trigger emits nothing. Message processing continues for diagnostics	Session reset only
`corrupt[sym]`	per symbol	table dup / set-full / qty-underflow, bad side on a tracked Add, book inconsistency (section 9)	Trigger emits nothing for that symbol	Session reset only

Flag	Scope	Set by	Effect	Cleared by
`tx_fault`	global	`TXF` write when full	Risk rejects every candidate with TX_FAULT	`CTRL.clear_fault`
`kill`	global	reset value is 1; host writes `CTRL.kill_switch`	Risk rejects every candidate	Host write 0

Why resync is not attempted: a lost or skipped handled message means the book may be missing an add or delete, so market state can no longer be proven exact. Framing itself resumes at the next frame (A1), but the book stays untrusted until session reset.

Session reset sequence (triggered by `CTRL.session_reset`, or by hardware `rst`):

1. `init_busy` goes high. All beats arriving while it is high are discarded and counted (`rx_dropped_init`).
2. The framer enters `SYNC_WAIT`: it discards beats until one with `tlast=1` has passed, then starts at the next beat.
3. The order table sweeps all sets to invalid at one set per cycle (`2^TABLE_AW` cycles). Ladders, hidden counts, `corrupt`, per-symbol `seq`, the message index and `session_invalid` clear in one cycle.
4. `init_busy` falls after the sweep plus 4 cycles.
5. Not cleared: counters (unless `CTRL.clear_counters`), kill, risk positions, configuration, `tx_fault`.

Caveat [N]: a reset is exact only at the start of a feed (A3). A mid-stream reset gives a book missing every order that was resting before it. The design does not detect that; it shows up as `ref_miss` counts, and the host owns the decision.

Every simulation vector starts with a sync frame: one beat, `tkeep=64'h3`, `tlast=1`, `tdata=0`. It is discarded by `SYNC_WAIT` and exists only to establish alignment.

4.5 Coding rules [N]

1. Synthesizable Verilog-2005. One module per file, file name = module name, prefix `t2t_`. `` `default_nettype none `` at the top of each file.
2. No latches, no `casex`, no tri-states, no `initial` blocks in `rtl/` (memories reset by the sweep, registers by `rst`).

3. No DSP: `(* use_dsp = "no" *)` on every arithmetic that could infer one. Memories larger than 64 bits x 32 use `(* ram_style = "block" *)`.
4. Instance names in `t2t_top` are normative because testbenches probe them by hierarchical path: `u_sess`, `u_ingest`, `u_framer`, `u_extract`, `u_symfilter`, `u_fa`, `u_ordtable`, `u_fb`, `u_book`, `u_trigger`, `u_risk`, `u_encoder`, `u_txf`, `u_regs`.
5. Every module file starts with a comment giving its spec section.

4.6 Numeric conventions [N]

All multi-byte ITCH and outbound fields are big-endian on the wire. Inside the core they are plain numbers. Price unit is 1/10000 USD, unsigned 32-bit. Stock is 8 ASCII bytes, space padded, character 0 in bits `[63:56]`. Side bit: 0 = bid (buy), 1 = ask (sell). Counters are 32-bit and saturate at `0xFFFFFFFF`.

5. Pipeline and buses

5.1 Stage table [N]

```
rx AXIS 512b -> [S1 ingest + ING fifo] -> [S2 framer] -> [S3 extract] -> [S4
symfilter]
  -> [FA fifo] -> [S5 ordtable] -> [FB fifo] -> [S6 book] -> [S7 trigger]
  -> [S8 risk] -> [S9 encoder + TXF fifo] -> tx AXIS 512b      (AXI-Lite:
u_regs, u_sess)
```

Stage	Module	Class	In	Out	Depth (cycles)	Spec
S1	`t2t_ingest`	input reg + ING	beat	beat	1 + 2	7.1
S2	`t2t_framer`	H1 behind ING, stoppable	beat	block (len, aligned bytes, idx)	4	7.2
S3	`t2t_extract`	H1	block	MSG bus	2	7.3
S4	`t2t_symfilter`	H1	MSG	FA bus	2	8.2
FA	`t2t_sfifo`	H2	FA bus	FA bus	2	4.3
S5	`t2t_ordtable`	H1, pops FA	FA bus	OP bus (1 or 2 ops)	4	8.3
FB	`t2t_sfifo`	H2	OP bus	OP bus	2	4.3
S6	`t2t_book`	H1, pops FB	OP bus	UPD signals	3	9
S7	`t2t_trigger`	H1	UPD	CAND	2	10.1
S8	`t2t_risk`	H1	CAND	DEC	4	10.2

Stage	Module	Class	In	Out	Depth (cycles)	Spec
S9	`t2t_encoder` + `TXF`	H1 + H2	DEC	tx beat	1 + 2	11.1

5.2 Buses [N]

MSG bus (S3 out), 320 bits. Fields not used by a type are 0.

Field	Bits	Meaning
`idx`	31:0	index of the block since session reset (counts every block, including skipped ones)
`type`	39:32	ASCII type
`locate`	55:40	stock locate
`ref`	119:56	order reference (for U: the original reference)
`new_ref`	183:120	U only
`side`	191:184	raw ASCII side byte, A and F only
`shares`	223:192	A, F: shares. E, C: executed. X: cancelled. U: new shares
`price`	255:224	A, F, U: price. C: execution price (informational)
`stock`	319:256	A, F only

FA bus, 325 bits = `{hit[0], sym[3:0], MSG}` with MSG in `[319:0]`, `sym` in `[323:320]`, `hit` in `[324]`.

OP bus (S5 out), 105 bits.

Field	Bits
`idx`	31:0
`qty`	63:32
`price`	95:64
`sym`	99:96
`side`	100
`code`	102:101 (0 ADD, 1 REDUCE, 2 CORRUPT)

Field	Bits
`remove`	103
`last`	104 (last op of this message)

UPD signals (S6 out), separate ports: `upd_valid`, `upd_sym[3:0]`, `upd_idx[31:0]`, `upd_last`, `upd_bid_px[31:0]`, `upd_bid_qty[31:0]`, `upd_ask_px[31:0]`, `upd_ask_qty[31:0]`, `upd_bid_lv[3:0]`, `upd_ask_lv[3:0]`, `upd_bid_stale`, `upd_ask_stale`, `upd_corrupt`, `upd_seq[31:0]`. `bid_ok` = `bid_lv != 0`, same for ask.

CAND signals (S7 out): `cand_valid`, `cand_sym[3:0]`, `cand_side` (0 buy, 1 sell), `cand_price[31:0]`, `cand_qty[15:0]`, `cand_rule[1:0]`, `cand_idx[31:0]`.

DEC signals (S8 out): the CAND fields plus `dec_valid`, `dec_pass`, `dec_reason[3:0]`.

5.3 Latency budget [N]

Definition: cycles from the cycle the beat holding a block's first byte is accepted at `rx_axis` (`tvalid`) to the cycle `tx_axis_tvalid` rises for the order that block caused, with all FIFOs empty and `tx_axis_tready=1`. Same path length for every handled type; a Replace adds 1 cycle to its second op.

Segment	Cycles
S1 (1 + ING 2)	3
S2 framer	4
S3 extract	2
S4 symfilter	2
FA	2
S5 ordtable	4
FB	2
S6 book	3
S7 trigger	2
S8 risk	4
S9 (1 + TXF 2)	3
Total	**31**

These are the design budget. T11 measures the real number into `reports/sim/latency_cycles.csv`. A stage that cannot meet its depth needs a spec change first (rule 4). Nanoseconds are never written down before a Vivado run; then ns = cycles x achieved period, labelled computed, excluding MAC, PHY, PCIe and network.

6. Wire formats

6.1 Inbound ITCH 5.0 [N after VERIFY-2 and VERIFY-9]

Offsets count from the type byte (offset 0), after the 2-byte length prefix. The length column is the stored length (prefix excluded). T1 checks every row against the NASDAQ TotalView-ITCH 5.0 document and fixes this table first if anything differs.

Type	Char	Len	Fields used (offset:bytes)
Add Order	`A`	36	locate 1:2, ref 11:8, side 19:1, shares 20:4, stock 24:8, price 32:4
Add Order with MPID	`F`	40	as A; attribution 36:4 ignored
Order Executed	`E`	31	locate 1:2, ref 11:8, executed shares 19:4 (match number 23:8 ignored)
Order Executed with Price	`C`	36	as E; printable 31:1 ignored; price 32:4 (informational)
Order Cancel	`X`	23	ref 11:8, cancelled shares 19:4
Order Delete	`D`	19	ref 11:8
Order Replace	`U`	35	original ref 11:8, new ref 19:8, shares 27:4, price 31:4

Tracking number (offset 3:2) and timestamp (5:6) are not carried [N]. Other types are skipped by length: S 12, R 39, H 25, Y 20, L 26, V 35, W 12, K 28, J 35, h 21, P 44, Q 40, B 19, I 50, N 20 (lengths listed for T1 to verify; the design does not depend on them).

Block rules [N]. `len` is the 16-bit big-endian prefix.

Condition	Action	Counter	Invalidates session
`len == 0`	skip the 2-byte prefix only	`n_zero_len`	no

Condition	Action	Counter	Invalidates session
`len > 50`	`frame_err`: drop the rest of the frame	`frame_err`	yes
handled type, `len` differs from the table	skip the block	`n_bad_len`	yes
unhandled type, `1 <= len <= 50`	skip the block	`n_other_skipped`	no
A or F with side not `B` or `S`	on a tracked symbol: `CORRUPT` op. Otherwise ignored	`n_bad_side`	no (symbol corrupt)
frame ends inside a block	`frame_err`	`frame_err`	yes
non-contiguous `tkeep`, or short `tkeep` on a non-last beat	drop the rest of the frame	`keep_err`	yes

6.2 Outbound OUCH-style Enter Order frame [N]

Fixed 48 bytes, big-endian, defined by this spec (D4). One frame per order.

Offset	Bytes	Field	Value
0	1	type	`0x4F` ('O')
1	4	user_ref	monotonic counter, first order = 1, reset only by hardware reset
5	1	side	`0x42` buy or `0x53` sell
6	4	quantity	`cand_qty` zero-extended
10	8	stock	the symbol slot's configured stock
18	4	price	`cand_price`
22	4	time_in_force	`0x00000000` (IOC)
26	1	display	`0x59`
27	1	capacity	`0x50`
28	1	intermarket sweep	`0x4E`

Offset	Bytes	Field	Value
29	4	minimum quantity	0
33	1	cross type	`0x4E`
34	1	customer type	`0x4E`
35	2	appendage length	0
37	4	trigger_idx	`cand_idx` (message index that caused the order)
41	1	rule_id	`cand_rule`
42	6	pad	0

AXIS framing: one beat, `tdata[8*i +: 8]` = byte i, bytes 48 to 63 are 0, `tkeep = 64'h0000_FFFF_FFFF_FFFF`, `tlast = 1`. [TBD-7] if the downstream stage wants other field values.

6.3 Vector and expected-output files [N]

One generator (`model/gen.py`) writes every scenario as a directory under `vectors/<name>/`. Large vectors are not committed; their SHA-256 goes in `reports/sim/`.

File	One line is	Format
`stim.hex`	one input beat	`<gap> <tlast> <tkeep16> <tdata128>`: `gap` decimal idle cycles before the beat; `tlast` one hex digit; `tkeep` 16 hex digits; `tdata` 128 hex digits, byte 63 leftmost, so `$fscanf("%h")` into `[511:0]` gives `tdata` directly
`ctrl.txt`	one AXI-Lite write	`<beat_index> W <addr3> <data8>` (hex): done before beat `beat_index` is presented. Session reset and kill are writes to `0x004`
`txready.txt`	the tx `tready` pattern	a string of `0` and `1`, one char per cycle, repeated cyclically. Absent means always 1
`exp_op1.txt`	one block, from extract	`idx8 type2 len4 flags2 locate4 ref16 newref16 side2 shares8 price8 stock16` (hex digits). flags: bit0 handled, bit1 bad_len, bit2 zero_len, bit3 bad_side

File	One line is	Format
`exp_op2.txt`	one op into FB	`idx8 code1 sym1 side1 remove1 last1 price8 qty8`
`exp_op3.txt`	one book update	`idx8 sym1 last1 bidpx8 bidqty8 askpx8 askqty8 bidlv1 asklv1 bidhid5 askhid5 flags2 seq8`; flags: bit0 bid_stale, bit1 ask_stale, bit2 corrupt, bit3 crossed
`exp_op4.txt`	one candidate	`idx8 sym1 side1 rule1 price8 qty4`
`exp_op5.txt`	one risk decision	`idx8 sym1 pass1 reason1 side1 price8 qty4`
`exp_op6.txt`	one outbound frame	96 hex digits, byte 0 first
`exp_counters.txt`	one final counter	`<id dec> <value dec>` using the ids in 11.3

Comparison rule [N]: row by row, exact match, in emission order. Timing is never compared except by the latency and throughput tests. Model and RTL both emit rows in message order.

7. Front end: ingest, framer, extract

7.1 `t2t_ingest` (S1) [N]

Port	Dir	Width	Meaning
`clk`, `rst`	in	1	
`i_tdata`	in	512	rx beat
`i_tkeep`	in	64	
`i_tvalid`, `i_tlast`	in	1	There is no `tready`. The kernel ties `rx_axis_tready` to 1
`i_init_busy`	in	1	from `u_sess`: discard while high
`i_pop`	in	1	framer takes the head beat
`o_valid`	out	1	head beat valid
`o_data`, `o_keep`, `o_last`	out	512, 64, 1	head beat

Port	Dir	Width	Meaning
`o_overflow`	out	1	pulse: a beat was dropped because `ING` was full
`o_dropped_init`	out	1	pulse: a beat was discarded because of `i_init_busy`

Behaviour: the beat is registered once, then written to `ING` (width 577, depth `ING_DEPTH`). If `ING` is full the beat is dropped, `o_overflow` pulses and `session_invalid` is set. Sizing rule: `ING_DEPTH >= ceil(max frame bytes / 64)`; max frame is assumed 1500 bytes, so 24 beats, profile value 64.

7.2 `t2t_framer` (S2) [N]

Port	Dir	Width	Meaning
`i_valid`, `i_data`, `i_keep`, `i_last`	in	1, 512, 64, 1	head of `ING`
`o_pop`	out	1	consume the head beat
`i_hold`	in	1	`FA.afull` or `FB.afull`; stops issuing
`i_init_busy`	in	1	
`o_blk_valid`	out	1	one block per cycle at most
`o_blk_idx`	out	32	block index
`o_blk_len`	out	8	stored length (0 to 50)
`o_blk_bytes`	out	400	bytes of the block after the prefix, byte 0 (type) in `[7:0]`; bytes at and beyond `len` are 0
`o_frame_err`, `o_keep_err`	out	1	pulses
`o_cnt_frame`, `o_cnt_beat`	out	1	pulses for counters

States: `SYNC_WAIT` (after any reset), `RUN`, `DROP_FRAME` (after an error: consume beats up to and including `tlast`, then return to `RUN`).

Behaviour:

1. The framer sees the frame as a byte stream: the valid bytes of its beats in order. Non-last beats must have `tkeep` all ones; the last beat must have `tkeep = 2^n - 1` with n >= 1 (`keep_err` otherwise).

2. At the stream position `p` it reads `len = {byte[p], byte[p+1]}`. If `p + 2 + len` is beyond the end of the frame, `frame_err`. If `len > 50`, `frame_err`.
3. It issues one block per cycle whenever the whole block (2 + len bytes) is present in the 2-beat window, or the frame end proves it complete, and `i_hold = 0`. A block may straddle two beats. A beat is popped when all its bytes have been consumed.
4. Blocks with `len == 0` are issued as blocks with `o_blk_len = 0`. Every block gets an index, increasing by 1.
5. A block of 2 + len <= 52 bytes starting at window offset <= 63 always ends inside the 128-byte window.
6. Implementation hint [I]: precompute, per byte offset b of a new beat, `next[b] = b + 2 + len(b)`; then the pointer chase is a 64:1 mux of 7-bit values per cycle, and the byte alignment is a two-stage barrel shifter (by 8-byte lane, then by byte). This is how the 1 block per cycle rate is meant to close timing.

Rate: 1 block per cycle while beats are available. Sustained input rate is therefore `f_clk x average block bytes x 8` bits per second (computed, not claimed). `ING` absorbs bursts; a sustained overload fills it and sets `ingest_overflow`.

7.3 `t2t_extract` (S3) [N]

Port	Dir	Width	Meaning
`i_blk_valid`, `i_blk_idx`, `i_blk_len`, `i_blk_bytes`	in	1, 32, 8, 400	from framer
`o_msg_valid`	out	1	handled type and correct length
`o_msg`	out	320	MSG bus (5.2)
`o_blk_flags`	out	4	for OP1: bit0 handled, bit1 bad_len, bit2 zero_len, bit3 bad_side (unused here, set by S5 view)
`o_cnt_type`	out	7	one-hot pulse per type A F E C X D U
`o_cnt_other`, `o_cnt_zero`, `o_cnt_badlen`	out	1	pulses

Behaviour: take `type = byte 0`. If it is one of the seven handled types and `len` equals the table length (6.1), assemble big-endian fields from the offsets in 6.1 into the MSG bus and raise `o_msg_valid`. A handled type with a wrong length raises `o_cnt_badlen` (which

sets `session_invalid`) and produces no message. Unhandled types and zero-length blocks produce no message. Skipped blocks still appear in OP1 through `o_blk_valid` and flags (both are real ports, unconnected at the top, so synthesis removes them).

Reference-mode cross-check (EX-10): `tb/sv/tb_refmode.sv` feeds the same bytes through `axis_512_to_8_adapter` and `rtl/ref/itch_decoder_ref.v`, and requires `shares` and `price` of every Add Order to equal the MSG bus values.

8. Symbol filter and order table

8.1 Symbol slot rule [N]

Slots 0, 1, 2 are `AAPL`, `SPY`, `MSFT` (8 bytes, space padded, case sensitive: `41 41 50 4C 20 20 20 20` for AAPL). Slots 3 to 7 are chosen once at T1 by `tools/symbol_census.py` run on the real sample file (A5):

1. Count Add Order (A) plus Add Order with MPID (F) messages per stock over the whole file.
2. Remove slots 0 to 2.
3. Take the 5 stocks with the highest counts; ties broken by ASCII ascending.
4. Write the census to `reports/sim/symbol_census.txt` with the file name and SHA-256, and the 8 chosen stocks to `rtl/t2t_symbols.vh`.

If the sample file lacks AAPL, SPY or MSFT, stop and ask the owner [TBD-1]. A slot is active when its configured stock is not all zero. If two slots hold the same stock the lowest index wins.

8.2 `t2t_symfilter` (S4) [N]

Port	Dir	Width	Meaning
`i_msg_valid`, `i_msg`	in	1, 320	MSG bus
`i_cfg_stock`	in	`N_SYMBOLS` x 64	flat, slot s in `[64*s +: 64]`
`o_valid`, `o_fa`	out	1, 325	FA bus
`o_cnt_hit`, `o_cnt_miss`	out	1	pulses

Behaviour: for types A and F, compare `stock` with every active slot (two 32-bit halves compared in the first stage, combined in the second). Hit: forward with `hit=1` and `sym` = slot. Miss: drop and pulse `o_cnt_miss`. For E, C, X, D, U: forward with `hit=0`, `sym=0`; the table (8.3) supplies the symbol. Depth 2.

8.3 `t2t_ordtable` (S5) [N]

Why it exists: E, C, X, D and U carry a reference and no price, side or symbol. The table maps reference to that data.

Structure. `2^TABLE_AW` sets, `TABLE_WAYS` ways per set, set index = `ref[TABLE_AW-1:0]`. Entry (134 bits): `valid` (1), `ref` (64, full), `sym` (4), `side` (1), `price` (32), `shares` (32). One set is one memory word (all ways read together). A7: 512 sets x 4 ways = 2048 orders, about 8 BRAM36 (estimate, Vivado decides).

Port	Dir	Width	Meaning
`i_valid`, `i_fa`	in	1, 325	popped from `FA` by the table when `FB.afull = 0`
`o_pop`	out	1	pop `FA`
`i_fb_afull`	in	1	
`i_init_busy`	in	1	sweep in progress
`o_op_valid`, `o_op`	out	1, 105	OP bus, written to `FB`
`o_cnt_*`	out	1 each	pulses: `ins`, `ref_miss`, `dup_ref`, `set_full`, `qty_underflow`, `same_set_stall`

Behaviour per message (all lookups by `ref`; a hit means a valid way with an equal `ref`):

Msg	Case	Table effect	Ops emitted
A, F	side not B or S	none	`CORRUPT(sym)`, last
A, F	`ref` already present	none; `dup_ref`	`CORRUPT(sym)`, last
A, F	set full (no invalid way)	none; `set_full`	`CORRUPT(sym)`, last
A, F	else	write lowest invalid way	`ADD(sym, side, price, shares, remove=0)`, last
E, C, X	miss	none; `ref_miss`	none
E, C, X	hit, `x > entry.shares`	none; `qty_underflow`	`CORRUPT(entry.sym)`, last
E, C, X	hit, `x == entry.shares`	invalidate way	`REDUCE(entry.price, x, remove=1)`, last

Msg	Case	Table effect	Ops emitted
E, C, X	hit, `x < entry.shares`	`entry.shares -= x`	`REDUCE(entry.price, x, remove=0)`, last
D	miss	none; `ref_miss`	none
D	hit	invalidate way	`REDUCE(entry.price, entry.shares, remove=1)`, last
U	orig miss	none; `ref_miss`	none (new ref is not inserted)
U	orig hit	invalidate old, then insert `new_ref` with the old `sym` and `side`, the new price and shares	op 1 `REDUCE(old price, old shares, remove=1)`, not last; op 2 `ADD(sym, side, new price, new shares)`, last. If the insert hits `dup_ref` or `set_full`: op 2 is `CORRUPT(sym)`, last

Here `x` is the message `shares` field. Old is invalidated before the new insert, so both may use the same set.

A reference miss is benign by construction: every Add of a tracked symbol either enters the table or raises `CORRUPT`, so a miss means the order belongs to an untracked symbol or predates the session. It is counted, never fatal.

Hazards and timing. Depth 4 (address, memory out, memory output register plus way compare, select and write). The table has no forwarding. A message whose set index equals the set index of any message still inside the table pipeline is held (`FA` is not popped) until that message has written back. Same-set back-to-back rate is therefore 1 per 4 cycles; distinct sets run at 1 per cycle. A Replace occupies two issue slots (one per sub-op, each with its own set index). `[I]` a bypass network for same-set forwarding is a later optimisation and would be a spec change.

Init sweep. While `i_init_busy` the table writes all-invalid words to sets 0 to `2^TABLE_AW - 1`, one per cycle, and pops nothing.

9. Order book (`t2t_book`, S6)

9.1 Policy [N]

Per symbol and side the book keeps the best `BOOK_N` (8) price levels, plus a count of **hidden orders**: live orders whose price is worse than the worst stored level. Hidden orders keep no price and no quantity here; the order table still knows them. Staleness is derived from this state, never set by hand.

State per `{sym, side}` (the key): `lv` (0 to `BOOK_N`), levels `L[0..lv-1]` each `{px[32], qty[32], cnt[16]}`, and `hidden[20]`. Bids are sorted strictly descending (index 0 is best), asks strictly ascending. State per symbol: `corrupt` (1 bit), `seq` (32 bits).

Invariants [N] (formal property FP-6 and the model both check these):

1. `L[0..lv-1]` are exactly the best `lv` price levels of the true ladder (a prefix of it).
2. Every hidden order is strictly worse than `L[lv-1]` (vacuous when `lv = 0`).
3. No stored level has `qty = 0` or `cnt = 0`; for each level `qty = 0` if and only if `cnt = 0`.
4. Prices strictly monotonic as above; entries at or beyond `lv` are 0.

From 1 and 2: when `lv > 0` the best price is exact. Derived flags:

Flag	Definition
`side_ok`	`lv != 0`
`side_stale`	`lv == 0 and hidden != 0` (best price unknown)
`crossed`	both sides ok and `bid_px[0] >= ask_px[0]`
`tradable(sym)`	both sides ok, not crossed, not `corrupt`, not `session_invalid`

9.2 Operations [N]

A better price means higher for bids and lower for asks. `worst = L[lv-1]`.

ADD(p, q)

1. If some `L[i].px == p`: `qty += q`, `cnt += 1`. A `cnt` overflow (16 bits) sets `corrupt`.
2. Else if `hidden == 0` and `lv < BOOK_N`: insert a new level `{p, q, 1}` at its sorted position, `lv += 1`.
3. Else if `lv > 0` and `p` is better than `worst.px`: insert at its sorted position. If `lv == BOOK_N`, evict `worst` first: `hidden += worst.cnt`. Otherwise `lv += 1`.
4. Else: `hidden += 1`.

REDUCE(p, d, remove)

1. If some `L[i].px == p`: if `L[i].qty < d` then `corrupt`. Else `qty -= d`; if `remove`, then `cnt -= 1` (if `cnt` is already 0, `corrupt`). If exactly one of `qty`, `cnt` is now 0, `corrupt`. If both are 0, delete the level and shift later levels up, `lv -= 1`.
2. Else if `hidden > 0` and (`lv == 0` or `p` is worse than `worst.px`): the order is hidden. If `remove`, `hidden -= 1`. A partial reduce changes nothing.

3. Else `corrupt` (an order exists in the table but not in the book).

CORRUPT: sets `corrupt[sym]`; no ladder change.

`corrupt` is sticky until session reset. Once it is set the ladders of that symbol may still change (so the diagnostic snapshot stays live) but `tradable` is false.

9.3 Worked example (N = 2, bid side) [N]

This is test BK-W; the model must reproduce it exactly.

Step	Op	Levels (px:qty) after	`lv`	`hidden`	Flags
1	ADD 100 q10	100:10	1	0	ok
2	ADD 99 q5	100:10, 99:5	2	0	ok
3	ADD 98 q7	100:10, 99:5	2	1	ok
4	ADD 101 q3	101:3, 100:10	2	2	ok (99 evicted, cnt 1)
5	REDUCE 101 q3 remove	100:10	1	2	ok, best 100 exact
6	REDUCE 100 q10 remove	none	0	2	stale
7	ADD 105 q1	none	0	3	stale (cannot insert)
8	REDUCE 99 q5 remove	none	0	2	stale
9	REDUCE 98 q7 remove	none	0	1	stale
10	REDUCE 105 q1 remove	none	0	0	empty, not stale

9.4 Pipeline and hazards [N]

Stage	Work
B0	pop `FB`; read the ladder word for `{sym, side}` and the per-symbol flags
B1	parallel compare of `p` against every level; equality and better-than vectors; operation case select
B2	shift and insert, evict and delete; write the ladder; update the symbol's top-of-book copy; emit UPD

Depth 3. **Same-key rule:** if the op at B0 has the same `{sym, side}` as the op in B1, hold it for one cycle (one bubble, counted in `book_same_key_bubbles`). The B1 input takes the ladder from the B2 output register when keys match, so the result of the previous op is always seen. Different keys (including the other side of the same symbol) issue every cycle.

Throughput gate (G4) [N]: at the book-core input (the `FB` output interface): ops alternating over all 8 symbols give exactly 1.0 op per cycle; ops all on one key give exactly 0.5 per cycle, which is the floor. The gate is 1.0 and >= 0.5 respectively.

UPD emission: one update per op, including CORRUPT. It carries the symbol's top-of-book for both sides after the op, flags, and `seq` (which increments by 1 per op applied to that symbol, starting at 0 after reset). `upd_last` is the op's `last` bit, so a Replace yields one non-last update (the delete) and one last update (the add); the trigger acts only on `last`.

Reset: during `init_busy` the book ignores `FB` and clears all ladders, hidden counts, `corrupt`, `seq` in one cycle.

Host view: `bk_tob_flat` (a flat vector, per symbol: 4 x 32 price and quantity bits, 2 x 4 level bits, 2 x 20 hidden bits, flags, 32-bit `seq`; slice macros in `t2t_defs.vh`) feeds the register file (11.2). Counters: `book_ops`, `book_same_key_bubbles`, `book_corrupt_events` (CORRUPT transitions 0 to 1), `book_hidden_adds`, `book_evictions`, `upd_total`, `upd_with_stale` (any side stale).

10. Trigger and risk

10.1 `t2t_trigger` (S7) [N]

Inputs: the UPD signals (5.2), `i_global_enable`, `i_session_invalid`, and a flat per-symbol config `i_tcfg` (slot s in `[120*s +: 120]`):

Field	Bits	Meaning
`en`	1	symbol enabled
`rule_mask`	4	bit i enables rule i
`imb_shift`	3	imbalance shift
`buy_limit`	32	CROSS_BUY threshold
`sell_limit`	32	CROSS_SELL threshold
`max_spread`	32	spread cap
`qty`	16	order quantity

Outputs: the CAND signals. State: `armed[sym][4]`, one bit per rule. Evaluation happens only on `upd_valid and upd_last`.

Precondition P = `global_enable` and `en` and `tradable(sym)` (9.1, using `upd_*` flags) and `ask_px - bid_px <= max_spread`.

Rule id	Name	Condition `cond_i`	Candidate
0	CROSS_BUY	`ask_px <= buy_limit`	buy at `ask_px`
1	CROSS_SELL	`bid_px >= sell_limit`	sell at `bid_px`
2	IMB_BUY	`bid_qty >= (ask_qty << imb_shift)` (39-bit compare)	buy at `ask_px`
3	IMB_SELL	`ask_qty >= (bid_qty << imb_shift)`	sell at `bid_px`

Firing is edge-triggered: `fire_i = P and rule_mask[i] and cond_i and not armed[i]`. After every evaluation `armed[i] <= P and cond_i` (so a rule re-arms once its condition or P goes false). If several rules fire, only the lowest rule id emits. `cand_qty = qty` from config. `cand_idx = upd_idx`. Nothing is emitted when P is false (FP-3). Depth 2.

The prices are the touch prices, so a candidate is an immediately marketable limit order. It is an IOC order (6.2).

10.2 `t2t_risk` (S8) [N]

Inputs: CAND signals; per-symbol config `i_rcfg` (slot s in `[161*s +: 161]`); global config; `i_kill`, `i_tx_fault`.

Per-symbol field	Bits	Meaning
`cfg_valid`	1	the host has configured this symbol
`max_qty`	16	
`max_notional`	48	in 1/10000 USD
`collar_abs`	32	allowed distance from `ref_px`
`ref_px`	32	reference price (host-set)
`max_pos`	32	limit on the absolute net position, in shares

Global: `max_gross` (32), `max_orders` (16), `win_shift` (5).

State: `pos[sym]` signed 32-bit net position, `gross` unsigned 32-bit (saturating), `win_id` and `win_cnt`. The state counts **orders passed**, not fills (nothing in this design sees fills). That is deliberately conservative.

Reason codes and priority. The first failing check in this order wins; pass is code 0.

Order	Code	Name	Rejects when
1	1	KILL	`i_kill = 1`
2	8	TX_FAULT	`i_tx_fault = 1`
3	15	UNKNOWN	`cfg_valid = 0`
4	2	QTY	`qty = 0` or `qty > max_qty`
5	3	NOTIONAL	`price * qty > max_notional`
6	4	COLLAR	`abs(price - ref_px) > collar_abs`
7	5	POS_SYM	`abs(pos + (buy ? qty : -qty)) > max_pos`
8	6	POS_GLOB	`gross + qty > max_gross`
9	7	RATE	`win_cnt >= max_orders` (so `max_orders = 0` rejects everything)

Rate window: `win_id = idx >> win_shift`; when `win_id` changes, `win_cnt` resets to 0. The window is counted in ITCH messages, not clock cycles, so the model needs no cycle accuracy [TBD-5: a time-based window].

Sequential consistency [N]. Decisions are made in candidate order and each sees the state left by every earlier candidate, with candidates arriving up to once per cycle. On pass: `pos`, `gross` and `win_cnt` update, and `dec_pass = 1`. On reject nothing changes. The two-stage LUT multiplier (32 x 16 bits, no DSP) and the other state-independent checks sit in the early stages; the state-dependent checks (POS_SYM, POS_GLOB, RATE) sit in the last stage so the state loop is one cycle [I]. Depth 4.

`kill` resets to 1 (11.2). A session reset leaves risk state alone.

11. Encoder, registers, counters, top level

11.1 `t2t_encoder` (S9) and `TXF` [N]

Inputs: DEC signals, the per-symbol stock (`i_cfg_stock`, same flat vector as 8.2), `i_tx_ready`. On `dec_valid and dec_pass` the encoder builds the 48-byte frame (6.2) in one registered stage, assigns `user_ref = ++user_ref_counter`, and writes it to `TXF` (width 512 + 64 + 1, depth `TXF_DEPTH`). Rejected decisions produce no frame. `tx_axis_tvalid` is the `TXF` head valid; the beat is held until `tx_axis_tready`.

If `TXF` is full at write time the frame is dropped, `tx_overflow` counts, and `tx_fault` is set. While `tx_fault = 1` risk rejects every candidate (TX_FAULT). The position state already counted the dropped order, which is the conservative direction. Cleared by `CTRL.clear_fault`.

11.2 `t2t_regs` AXI-Lite register file [N]

Port names follow the upstream kernel stub: `s_axi_ctrl_awaddr[11:0]`, `awvalid`, `awready`, `wdata[31:0]`, `wstrb[3:0]`, `wvalid`, `wready`, `bresp[1:0]`, `bvalid`, `bready`, `araddr[11:0]`, `arvalid`, `arready`, `rdata[31:0]`, `rresp[1:0]`, `rvalid`, `rready`.

Protocol: a write is accepted when both `awvalid` and `wvalid` are high in the same cycle; `bvalid` rises the next cycle and holds until `bready`. A read is accepted on `arvalid`; `rvalid` rises the next cycle and holds until `rready`. One transaction outstanding per channel. `bvalid` is never high without a write (this fixes upstream finding F11). `wstrb` is honoured per byte lane. Response is OKAY (0) on success; SLVERR (2) on a write to a read-only register, an unmapped address, or a rejected write (below). Reads of unmapped addresses return `0xDEADBEEF` with OKAY, as upstream did. Coherence: registers marked S read a shadow that is latched all at once by `CTRL.snapshot`; before the first snapshot the shadow is 0.

Address	Name	Access	Reset	Meaning
`0x000`	ID	RO	`0x74320001`	magic and version
`0x004`	CTRL	RW	`0x2`	bit0 `session_reset` (write 1, self-clears); bit1 `kill_switch` (level); bit2 `clear_counters` (W1); bit3 `global_enable` (level); bit4 `snapshot` (W1); bit5 `clear_fault` (W1)
`0x008`	STATUS	RO		bit0 `session_invalid`; bit1 `init_busy`; bit2 `tx_fault`; bit3 `kill_active`
`0x00C`	CAPS	RO		`[3:0]` `N_SYMBOLS-1`; `[7:4]` `BOOK_N`; `[15:8]` `TABLE_AW`; `[19:16]` `TABLE_WAYS`
`0x010`	BEST_BID0	RO live	0	slot 0 best bid price (upstream-compatible); 0 if no bid
`0x018`	BEST_ASK0	RO live	0	slot 0 best ask price (upstream-compatible)

Address	Name	Access	Reset	Meaning
`0x100 + 4k`	COUNTER k	RO S	0	k = 0 to 63 (11.3)
`0x200 + 0x20*s`	BOOK s	RO S	0	+0 bid_px, +4 bid_qty, +8 ask_px, +C ask_qty, +10 flags, +14 bid_hidden, +18 ask_hidden, +1C seq. flags: bit0 bid_ok, 1 ask_ok, 2 bid_stale, 3 ask_stale, 4 corrupt, 5 crossed, `[11:8]` bid_lv, `[15:12]` ask_lv
`0x400 + 0x40*s`	SYM s	RW	see below	per-symbol configuration
`0x800`	RISK_G	RW	0	+0 max_gross, +4 gross (RO S), +8 max_orders `[15:0]`, +C win_shift `[4:0]`

Symbol block (offsets from `0x400 + 0x40*s`): `+0x00` stock chars 0 to 3 (char 0 in bits 31:24); `+0x04` stock chars 4 to 7; `+0x08` sym_ctrl (bit0 `en`, bit1 `cfg_valid`, `[7:4]` rule_mask, `[10:8]` imb_shift); `+0x0C` buy_limit; `+0x10` sell_limit; `+0x14` max_spread; `+0x18` order_qty `[15:0]`; `+0x1C` max_qty `[15:0]`; `+0x20` max_notional low 32; `+0x24` max_notional high `[15:0]`; `+0x28` collar_abs; `+0x2C` ref_px; `+0x30` max_pos; `+0x34` pos (RO S). Reset: stock = the `rtl/t2t_symbols.vh` default for that slot; everything else 0, so `cfg_valid = 0` and `en = 0` (fail-closed).

Rejected writes (SLVERR, no effect): stock registers while `global_enable = 1`; any field written with non-zero bits above its width.

11.3 Counter ids [N]

Id	Name	Id	Name
0	rx_beats	24	tbl_qty_underflow
1	rx_frames	25	tbl_same_set_stalls
2	ingest_overflow	26	book_ops
3	rx_dropped_init	27	book_same_key_bubbles
4	frame_err	28	book_corrupt_events
5	keep_err	29	book_hidden_adds

Id	Name	Id	Name
6	blocks_total	30	book_evictions
7 to 13	n_A, n_F, n_E, n_C, n_X, n_D, n_U	31	trig_cand
14	n_other_skipped	32	risk_pass
15	n_zero_len	33 to 40	risk_rej for codes 1, 2, 3, 4, 5, 6, 7, 8
16	n_bad_len	41	risk_rej code 15
17	n_bad_side	42	tx_frames
18	sym_hit	43	tx_overflow
19	sym_miss	44	upd_total
20	tbl_ins	45	upd_with_stale
21	tbl_ref_miss	46	fatal_overflow
22	tbl_dup_ref	47 to 63	reserved, read 0
23	tbl_set_full		

11.4 Top level [N]

`rtl/t2t_top.v`: ports `clk`, `rst`, `rx_axis_tdata[511:0]`, `rx_axis_tkeep[63:0]`, `rx_axis_tvalid`, `rx_axis_tlast`, `tx_axis_tdata[511:0]`, `tx_axis_tkeep[63:0]`, `tx_axis_tvalid`, `tx_axis_tready`, `tx_axis_tlast`, and the AXI-Lite port set of 11.2. It instantiates the modules with the instance names in 4.5. `u_sess` owns `init_busy`, `session_invalid`, the session reset sequence and the sweep timing (4.4).

`rtl/t2t_kernel.v`: same port list and widths as upstream `hft_trading_kernel` (`ap_clk`, `ap_rst_n`, `rx_axis_*` including `rx_axis_tready`, `tx_axis_*`, `s_axi_ctrl_*`). It converts reset, ties `rx_axis_tready` to 1, and instantiates `t2t_top`. This keeps a later Vitis packaging step mechanical. Out-of-context implementation uses `t2t_top` as the synthesis top.

12. Verification catalogue

12.1 Method [N]

The Python model in `model/` is bit-accurate for everything in 6.3 and implements sections 7 to 11 as written (framer, extract, filter, table, book with the 9.2 operations, trigger, risk, encoder, counters). It has no clock. `gen.py` writes `stim.hex`, `ctrl.txt`, `txready.txt`

and every `exp_*` file. Three harnesses consume the same files: the SV file-driven testbench (directed, regression, soak), cocotb (constrained random, protocol corners, backpressure), and the soak runner (Verilator, same SV bench). Testbenches probe internal nets by the instance paths in 4.5. All results go to `reports/sim/<test-id>.log` with the vector name and SHA-256 at the top.

Tx backpressure scenarios keep `TXF` from filling, so they stay bit-comparable. The `tx_fault` path has its own test (EN-3) with no model comparison beyond the counters.

Test ids are `<family>-<n>`. A row lists the sub-cases a to z; each is one vector.

12.2 Tests

Id	Module	Sub-cases and pass criterion	Task
CHAR-1	upstream	`tb_axis_adapter.v` unmodified: prints PASS, 138 bytes	T0
CHAR-2	upstream	`tb_itch_decoder_real_data.v` unmodified: runs to `$finish` at 7520 ns. A new instrumented bench logs every `order_valid` pulse, shares and price. Record the result either way (VERIFY-4)	T0
CHAR-3	`sim_data.hex`	`tools/audit_sim_data.py` prints byte count, block count, stride, length field, types, stock bytes (VERIFY-6)	T0
MDL-1	model	each message type from hand-built examples taken from the ITCH document; 6.1 table verified (VERIFY-2, 9)	T1
MDL-2	model	book invariants (9.1) hold after every op over random streams; BK-W reproduced	T1
MDL-3	gen	legacy vector regenerated with valid 38-byte framing; same 20 prices and shares	T1

Id	Module	Sub-cases and pass criterion	Task
FR-1	framer	a) one block per beat; b) several blocks per beat; c) block straddling two beats; d) block ending exactly at a beat end; e) partial last `tkeep`; f) zero-length block; g) unhandled type; h) `len` 51 gives `frame_err`, rest of frame dropped, next frame decoded; i) non-contiguous last `tkeep` gives `keep_err`; j) short `tkeep` mid-frame gives `keep_err`; k) overload fills `ING`: `ingest_overflow`, `session_invalid`; l) `i_hold` for many cycles loses nothing; m) reset in the middle of a burst, then `SYNC_WAIT` realigns. 0 mismatches vs OP1 and counters	T2
EX-1	extract	a to g) one per handled type, every field; h) known type with wrong length gives `n_bad_len` and `session_invalid`; i) EX-10 reference-mode cross-check on all Add messages	T3
SF-1	symfilter	a) a hit on each of 8 slots; b) near misses (`AAPL` with other padding, lower case) do not hit; c) non-add types always pass; d) duplicate slot stock: lowest wins; e) unused slot (all-zero stock) never hits	T4
TB-1	table	a) add then execute partial then execute rest; b) cancel partial; c) delete; d) replace; e) benign miss; f) duplicate ref gives `CORRUPT`; g) set full (5 refs in one set) gives `CORRUPT`; h) execute more than resting gives `CORRUPT`; i) same-set back-to-back obeys the hold and the order of effects; j) sweep after session reset; k) replace where old and new share a set	T5
BK-1	book	a) sorted insert both sides; b) merge at same price; c) fill beyond N, eviction to hidden; d) delete best with hidden; e) delete to empty with hidden gives stale; f) recovery from stale when hidden reaches 0; g) delete a middle level; h) replace across levels; i) add worse than worst; j) inconsistent op gives `corrupt`; k) `corrupt` survives until session reset; l) N=2 random stress against the model; m) forwarding, back-to-back same key random; n) BK-W worked example	T6

Id	Module	Sub-cases and pass criterion	Task
TH-1	book	alternating over 8 symbols: exactly 1.0 op per cycle over at least 100000 ops	T6
TH-2	book	one key only: exactly 0.5 op per cycle	T6
TR-1	trigger	a) each rule fires; b) priority; c) edge re-arm; d) blocked by stale; e) blocked by `corrupt`; f) blocked by crossed; g) blocked by spread; h) blocked by `global_enable = 0`; i) a Replace's first op causes no candidate	T7
RK-1	risk	a to h) each limit one below, at, one above; i) kill; j) unknown config; k) reason priority with several failures; l) back-to-back candidates see each other's state; m) rate window roll-over; n) `max_orders = 0`	T8
EN-1	encoder	frame bytes equal OP6 exactly	T9
EN-2	encoder	`tready` low for fewer cycles than `TXF` holds: no loss, order kept	T9
EN-3	encoder	`TXF` overflow: `tx_overflow`, `tx_fault`, later candidates rejected TX_FAULT, `clear_fault` recovers	T9
RG-1	regs	a) read and write every register; b) SLVERR cases; c) snapshot coherence; d) stock write refused while enabled; e) `kill` is 1 after reset; f) offsets `0x010` and `0x018`	T9
IT-1	top	directed end-to-end scenario set: 0 mismatches at OP1 to OP6 and counters	T9
IT-2	top	regenerated legacy vector (MDL-3) end to end	T9
IT-3	top	reset in the middle of a burst, then a fresh session	T9
IT-4	top	cocotb constrained random, at least 20 seeds, all pass; failing seeds logged	T10
SO-1	top	1,000,000 synthetic messages, mixed types, deep books, replace chains, paced at no more than 0.8 message per cycle: 0 mismatches, `ingest_overflow = 0`	T11

Id	Module	Sub-cases and pass criterion	Task
SO-2	top	first 1,000,000 blocks of the real NASDAQ file: counts per type agree between model, RTL and the independent tool (G3). Records `corrupt` events and tradable fraction	T11
TH-3	top	end-to-end messages per cycle on SO-1 and SO-2; sustainable line rate computed	T11
LA-1	top	cycles from 5.3 per path (Add, Execute, Delete, Replace), empty FIFOs	T11
FP-1 to FP-6	formal	see 12.3	T6 to T12
COV-1	all	coverage report (12.4)	T12
MUT-1	all	mutation report (12.4)	T12
VIV-1 to 3	vivado	synthesis, implementation, parse (13)	T13

12.3 Formal properties (SymbiYosys) [N]

Id	Property	Target
FP-1	Risk never passes an order whose quantity, notional, collar, position, gross or rate violates the limits, and `kill = 1` always rejects	unbounded (`abc pdr`)
FP-2	`corrupt[sym]` is sticky: once set it clears only on session reset	unbounded
FP-3	Trigger emits nothing when P is false (stale, corrupt, crossed, spread, disabled, session invalid)	unbounded
FP-4	Framer conserves bytes: every accepted byte is issued once or counted as skipped	unbounded, else bounded with depth stated
FP-5	`ING` , `FA` , `FB` , `TXF` neither drop nor duplicate a beat under any push and pop pattern (except `ING` overflow, which is counted)	unbounded

Id	Property	Target
FP-6	Book invariants 9.1 items 3 and 4 hold in every reachable state (bid levels strictly descending, asks ascending, no zero-quantity level, entries beyond `lv` zero)	unbounded, else bounded with depth stated

12.4 Coverage and mutation [N]

Verilator `--coverage-line` and `--coverage-toggle` over `rtl/` (testbenches excluded), merged across all regression runs. FSM state coverage is measured as functional bins (`SYNC_WAIT`, `RUN`, `DROP_FRAME`, and each ladder case of 9.2) plus the cross of message type x book state x stale state x backpressure. Branch coverage is not claimed. Unreached lines or toggles get a test or a written waiver in `reports/cov/waivers.md`.

Mutation: `mcy` generates mutants of `rtl/`, the full regression is the oracle. Kill rate = killed / non-equivalent mutants. Surviving mutants are either listed as equivalent with a reason or answered with a new test. Both lists are published.

13. Gates and evidence

13.1 Gates [N]

Id	Gate	Pass criterion	Evidence file	Label	Produced by
G1	Functional correctness	0 mismatches model vs RTL at OP1 to OP6 and counters on every directed and random vector	`reports/sim/mismatch.txt`	measured	here (T2 to T10)
G2	Soak	SO-1: 0 mismatches, 0 `ingest_overflow`, 0 `fatal_overflow`	`reports/sim/soak.log` + vector hash	measured	here (T11)
G3	Independent ITCH cross-check	SO-2 on a real NASDAQ ITCH 5.0 file: per-type message counts and per-symbol Add counts agree between model, RTL and an independent open-source ITCH 5.0 parser [TBD-4: tool and version]. File name and SHA-256 recorded	`reports/sim/itch_xcheck.txt`	measured	here, file from the owner if there is no network
G4	Throughput at the book-core input	TH-1 = 1.0 op/cycle (alternating symbols); TH-2 >= 0.5 op/cycle (same key). End-to-end TH-3 is reported, not gated	`reports/sim/throughput.csv`	measured	here (T6, T11)
G5	Latency in cycles	LA-1 per path, compared with the 5.3 budget; any difference is stated	`reports/sim/latency_cycles.csv`	measured	here (T11)
G5n	Latency in ns	cycles x achieved period, excludes MAC, PHY, PCIe, network	`reports/derived/latency_ns.csv`	computed	here, after G6

Id	Gate	Pass criterion	Evidence file	Label	Produced by
G6	Fmax (A7)	Sweep 5.000 then 6.667 ns, post-route, out of context. Report the best period with WNS >= 0. Status: `MET` if 5.000 closes; `MISSED, closes at 6.667` if only the fallback closes; `MISSED` if neither. The number is reported whatever the status	`reports/vivado/timing_summary_<period>.rpt`	reported	you run, I parse
G6u	Fmax (U55C)	4.000 ns with WNS >= 0. Only if a U55C run exists	`reports/vivado/...`	reported	you
G7	LUT, FF, BRAM	reported, no threshold unless set	`reports/vivado/utilization.rpt`	reported	you run, I parse
G7e	Yosys counts	LUT and FF counts until Vivado replaces them	`reports/synth/yosys_stat.txt`	estimate	here
G8	Coverage	line >= 90% and toggle >= 90% over `rtl/`, waivers written; FSM and ladder-case bins 100% reached. Branch is not claimed	`reports/cov/`	measured	here (T12)
G9	Formal	FP-1, 2, 3, 5 unbounded; FP-4, 6 unbounded or bounded with the depth stated	`reports/formal/*.log`	bounded-proven	here (T12)
G10	Mutation	>= 90% of non-equivalent mutants killed	`reports/mutation/summary.json`	measured	here (T12)
G11	Evidence hygiene	`reports/MANIFEST.json` lists every report with SHA-256, commit hash, tool versions and label; every README figure links to it	`reports/MANIFEST.json`	n/a	T13

A gate that fails is reported with the real number and a fix task is opened (rule 7). G8 uses 90% because Verilator reports line and toggle only; the plan's 95% line and branch figure is withdrawn rather than claimed on a metric that was not measured.

13.2 Labels [N]

Exactly one label per number.

Label	Means	Qualifier required
measured	produced by a simulation run in this repo	vector name and hash
bounded-proven	a SymbiYosys result	`(depth N)` for BMC, `(unbounded, pdr)` for a PDR proof
reported	read from a Vivado report you ran	Vivado version, part, constraint
computed	derived by a script from other labelled numbers	the inputs
estimate	Yosys or any non-Vivado count	tool and version

13.3 README rules [N]

Every number links to a file in `reports/` and a commit hash. The README says the implementation is out of context, which is optimistic compared with a full shell. It says nothing about U55C results unless G6u ran. The resume line for latency reads: latency in ns (computed, excludes MAC and PHY).

14. Repository, build, tooling

14.1 Layout [N]

```
tick2trade-fpga/
  README.md  CREDITS.md  LICENSE  Makefile
  docs/SPEC.md                    this document, exported
  upstream/                       byte-for-byte upstream files + UPSTREAM.md
(no license text added)
  rtl/
    t2t_defs.vh  t2t_symbols.vh
    t2t_top.v  t2t_kernel.v  t2t_sess.v  t2t_regs.v
    t2t_ingest.v  t2t_sfifo.v  t2t_framer.v  t2t_extract.v  t2t_symfilter.v
    t2t_ordtable.v  t2t_book.v  t2t_trigger.v  t2t_risk.v  t2t_mul32x16.v
t2t_encoder.v
    ref/itch_decoder_ref.v        derived from upstream, defect fixed,
reference mode only
  model/   itch.py framer.py table.py book.py trigger.py risk.py encoder.py
gen.py replay.py
  tools/   audit_sim_data.py symbol_census.py extract_itch.py itch_xcheck.py
VERSIONS.txt
  vectors/ legacy/ directed/ (small, committed)   soak/ real/ (not committed;
hashes in reports)
  tb/sv/   tb_top.sv tb_book_core.sv tb_refmode.sv tb_char_upstream.sv  ...
  tb/cocotb/
  formal/  *.sby and property modules
  vivado/  synth_ooc.tcl impl_ooc.tcl t2t.xdc
  scripts/ parse_reports.py
  reports/ sim/ cov/ formal/ mutation/ vivado/ synth/ derived/ MANIFEST.json
```

14.2 Makefile targets [N]

`make audit` (T0 characterisation), `make sim` (directed, regression), `make soak`, `make cov`, `make formal`, `make mutation`, `make vivado` (you run it; writes `reports/vivado/`). `make vivado` accepts `PERIOD=5.000` and sweeps the list in 4.1 when omitted. All targets write a log and a hash into `reports/` and fail on any mismatch.

14.3 Tools

Verilator 5.x (`--binary --timing` for the SV benches, `--coverage-line --coverage-toggle`), Icarus for plain Verilog tests, Python 3.10 or newer, cocotb, Yosys, SymbiYosys, mcy, Vivado (your machine). T0 records the installed versions in `tools/VERSIONS.txt` , checks that Verilator offers line and toggle coverage (VERIFY-8) and that the free Vivado edition accepts `xc7a35tcpg236-1` (VERIFY-7). CI skeleton (GitHub Actions): `audit` , `sim` , `formal` ; never Vivado.

15. Tasks

One task per change set. Each task is done only when its listed tests pass and their logs are in `reports/sim/` (rule 5). Tasks needing the owner are marked.

Task	Deliverable	Tests and gates	Needs
T0	Repo skeleton, `upstream/` copy with `UPSTREAM.md` (hash, SHA-256s), `CREDITS.md` , `LICENSE` for new files, Makefile, CI skeleton, `tools/VERSIONS.txt` , characterisation benches	CHAR-1, CHAR-2, CHAR-3 logged; VERIFY-4, 5, 6, 7, 8 closed (confirmed or corrected in this spec)	Owner: run `get_parts xc7a35t*` ; send the license request (3.3)
T1	`model/` (all modules), `gen.py` , vector formats, 6.1 checked against the ITCH document, real sample file obtained, symbol census and `t2t_symbols.vh`	MDL-1, MDL-2, MDL-3; VERIFY-2, 3, 9 closed; TBD-1 resolved	Owner supplies the sample file if there is no network
T2	`t2t_sfifo` , `t2t_ingest` , `t2t_framer`	FR-1; FP-4, FP-5 on the FIFOs	
T3	`t2t_extract` , `rtl/ref/itch_decoder_ref.v` , `tb_refmode.sv`	EX-1 (including EX-10)	
T4	`t2t_symfilter`	SF-1	
T5	`t2t_ordtable` , `FA` , `FB` wiring	TB-1	
T6	`t2t_book`	BK-1, TH-1, TH-2 (G4); FP-6, FP-2	
T7	`t2t_trigger`	TR-1; FP-3	
T8	`t2t_risk` , `t2t_mul32x16`	RK-1; FP-1	

Task	Deliverable	Tests and gates	Needs
T9	`t2t_encoder`, `t2t_regs`, `t2t_sess`, `t2t_top`, `t2t_kernel`	EN-1 to 3, RG-1, IT-1, IT-2, IT-3	
T10	cocotb suite	IT-4	
T11	Soak runner, real-file replay, latency and throughput tables	SO-1, SO-2, TH-3, LA-1 (G2, G3, G4, G5)	
T12	Coverage closure, all formal runs, mutation campaign	COV-1 (G8), FP-1 to 6 (G9), MUT-1 (G10)	
T13	Vivado scripts, XDC, `parse_reports.py`, README, `MANIFEST.json`	G6, G7, G5n, G11	Owner runs `make vivado` and commits `reports/vivado/`

If the Vivado run misses the A7 target, the critical stage is named from the report and re-pipelined in a new task; the claim is never adjusted instead.

Task map from the approval plan: old T0 to T3 unchanged; old T4 became T4 and T5; old T5 to T12 became T6 to T13.

16. Registers of open items

16.1 [VERIFY] register

Id	Fact to confirm	How	Task	Status
VERIFY-1	A1: the stream is MoldUDP64 payload as back-to-back [len16][message] blocks, frames hold whole blocks	read how the upstream adapter and top are fed; check the real file framing	T0, T1	open
VERIFY-2	A4: longest stored message is 50 bytes; the length column of 6.1	NASDAQ TotalView-ITCH 5.0 specification	T1	open
VERIFY-3	A5: a real ITCH 5.0 sample file can be downloaded	try NASDAQ's public ITCH sample area; else owner supplies	T1	open

Id	Fact to confirm	How	Task	Status
VERIFY-4	F2, F3: upstream `itch_decoder` never raises `order_valid` and never captures price byte 0	CHAR-2 instrumented bench	T0	open
VERIFY-5	upstream commit hash, absence of a LICENSE file	`git rev-parse HEAD`, file listing	T0	open
VERIFY-6	F10: `sim_data.hex` is 20 blocks of 36 bytes with length field 36	CHAR-3	T0	open
VERIFY-7	The free Vivado edition accepts `xc7a35tcpg236-1`	`get_parts xc7a35t*`	T0	open, owner
VERIFY-8	Verilator offers line and toggle coverage	`verilator --help`, a dry run	T0	open
VERIFY-9	Field offsets in 6.1	ITCH document	T1	open
VERIFY-10	A6: byte lane 0 is the first byte on the wire for CMAC rx and tx	Xilinx PG203	T0	open

16.2 [TBD] register (owner choices)

Id	Question	Default used until answered
TBD-1	Symbols for slots 3 to 7	the 8.1 rule on the sample file; stop and ask if AAPL, SPY or MSFT is missing
TBD-2	Which sample file and day	the first one VERIFY-3 can obtain
TBD-3	License for new files: MIT or Apache-2.0	MIT
TBD-4	Independent open-source ITCH tool for G3, and its pinned version	chosen at T1; must not be code from this project
TBD-5	Rate limit by time instead of by messages	by messages (10.2)
TBD-6	U55C table size and whether to use URAM	`TABLE_AW=12`, `TABLE_WAYS=4`; no claims

Id	Question	Default used until answered
TBD-7	Outbound field values the downstream stage expects	6.2 constants
TBD-8	Carry the ITCH timestamp and tracking number	not carried

16.3 C-choices made in this detailing pass (confirm or change)

These were not in the approved text. They are normative until you change them, and a change is made here first.

Id	Choice	Where
C1	Derived `stale` (`lv = 0` with hidden orders); best price exact whenever `lv > 0`	9.1
C2	Separate sticky `corrupt` per symbol, cleared only by session reset	4.4, 9
C3	Set-associative order table (4 ways), never overwrites, a full set is `CORRUPT`	8.3
C4	Elastic points are only `ING`, `FA`, `FB`, `TXF`, with credit flow control	4.3
C5	Trigger rules, edge-triggered, fixed priority, touch-price IOC candidates	10.1
C6	Risk is sequentially consistent; message-count rate window; exposure counts orders passed; `kill` resets to 1	10.2, 11.2
C7	`session_invalid` on any lost handled message; resync only by session reset	4.4
C8	`tx_fault` on `TXF` overflow, cleared by `CTRL.clear_fault`	11.1
C9	Outbound frame constants and the 48-byte length	6.2
C10	Latency budget of 31 cycles	5.3
C11	Register map, counter ids	11.2, 11.3
C12	Vector file formats and observation points	6.3

Id	Choice	Where
C13	A fixed copy of the upstream decoder as reference mode	3.4, T3
C14	Label qualifier for PDR proofs; A7 Fmax status words	13.1, 13.2
C15	Task numbering T0 to T13	15